

Turing Way-Aligned Readiness Snapshot

This is not an official Turing Way certification. The Turing Way is public best-practice guidance, not a certifying authority or institutional policy. This document is a transparent, evidence-based readiness snapshot produced by scoring ten observable-evidence criteria against practices described in *The Turing Way*.

- **Repository reviewed:** `stjude-biohackathon/KIDS25-Team21`
- **Commit reviewed:** `be2e0cff95257987204cc92f1a0d00c63fa23def` (branch `main`)
- **Review date:** 2026-09-18
- **Scope:** Entire public repository as checked out at the commit above — `AMPLIFICATION/`, `CODE/`, `CODE_deprecated/`, `NUCLEIC_ACID_DELIVERY/`, `NUCLEIC_ACID_PURIFICATION/`, and root-level files.
- **Confidence:** Medium. The repository is small (7 commits, ~18 tracked files) so file-presence checks are exhaustive, but no build, test, or CI run was executed, and PDF/DOCX protocol contents were not opened.

Method

Following the `turing-way-certified` skill, this review used the required Turing Way learning-assistant tools before scoring: `get_turing_way_review_evidence` retrieved background guidance for project design, reproducibility, and version control/collaboration; `list_resources` and `get_resource` retrieved eight pinned Turing Way chapters; and `get_turing_way_evidence_packets` resolved nine citation-safe evidence packets pairing each repository observation with a pinned Turing Way source. Repository facts below were obtained by direct inspection of the working tree and `git log / git tag` output — no repository files were modified.

Score summary

#	Criterion	Score (/10)	Key evidence
1	Project purpose and scope	1	No root README, scope statement, or intended-user description; only a one-line <code># KIDS25-Team21</code> heading in <code>CODE_deprecated/README.md</code> .
2	Version control and provenance	4	Git history exists (7 commits on <code>main</code>) and is traceable, but there are no tags/releases and several commit messages are non-descriptive (<code>commit</code> , <code>Add files via upload</code>).
3	Open collaboration	1	No <code>CONTRIBUTING.md</code> , <code>CODE_OF_CONDUCT.md</code> , or <code>.github/issue/PR</code> templates anywhere in the tree.
4	Reproducible environments	2	<code>CODE/requirements.txt</code> lists dependencies but none are version-pinned; no <code>environment.yml</code> , <code>Dockerfile</code> , or Python-version file exists.
5	Data and workflow provenance	1	Vendor protocol PDFs/DOCX are stored under topic folders with no index, metadata, schema, or documented provenance/licensing for each source.
6	Testing and validation	0	No test files, test directory, or documented validation procedure exist for the <code>CODE/</code> chatbot pipeline.
7	Automation and continuous integration	0	No <code>.github/workflows/</code> directory or any other CI configuration is present.
8	Documentation and usability	2	<code>CODE/*.py</code> modules have short class-level docstrings, but there is no usage guide, onboarding path, or documented limitations.
9	Licensing, attribution, and responsible reuse	3	An MIT <code>LICENSE</code> exists only at <code>CODE_deprecated/LICENSE</code> ; no license covers the repository root or the active <code>CODE/</code> , <code>AMPLIFICATION/</code> ,

#	Criterion	Score (/10)	Key evidence
			NUCLEIC_ACID_DELIVERY/ , or NUCLEIC_ACID_PURIFICATION/ content. No CITATION.cff .
10	Ethics, accessibility, and sustainability	0	No ethical/privacy considerations, accessibility notes, or maintenance/archival plan were found anywhere in the repository.

Total: 14 / 100

Evidence and citations

Criterion	Repository observation	Turing Way guidance cited
Project purpose and scope	CODE_depricated/README.md is the only README, and it contains just a project-name heading.	Project Documentation — a README should introduce and explain what a project is about.
Version control and provenance	Repository tree at be2e0cf shows 7 commits and no tags.	Version Control — version control provides provenance information and an auditable, reproducible version history.
Open collaboration	Repository tree at be2e0cf has no CONTRIBUTING.md / CODE_OF_CONDUCT.md / issue templates.	Guide for Collaboration — projects intending outside contribution should include contributing guidelines and a code of conduct.
Reproducible environments	CODE/requirements.txt lists unpinned packages with no environment capture file.	Project Design — Methods — dependency managers should keep and pin the exact versions used in development.
Data and workflow provenance	NUCLEIC_ACID_PURIFICATION/ stores vendor PDFs with no index or documented provenance.	Research Data Management Checklist — teams should document data sources, standardise practices, and set up a data management plan.
Testing and validation	CODE/ contains only application modules, no tests.	Continuous Integration — Best Practices — testing should be automated so regressions are caught early.
Automation and CI	Repository tree at be2e0cf has no .github/workflows/ or other CI config.	Continuous Integration — Best Practices — CI integrates and validates changes frequently, surfacing failures early.
Documentation and usability	CODE/chatbot.py and sibling modules have only short class docstrings; no usage guide exists.	Project Documentation — project documentation should include a README, roadmap, and onboarding information.
Licensing, attribution, and reuse	CODE_depricated/LICENSE (MIT) is not present at the repository root.	Licensing — a license file should be placed in the top-level directory so it is picked up and displayed by the host.
Ethics, accessibility, and sustainability	Repository tree at be2e0cf has no ethics, accessibility, or maintenance statement.	Guide for Ethical Research — projects benefit from planning ethical considerations into project design regardless of institutional requirements.

Recommendations

1. Add a root-level `README.md` stating the project's purpose (a biohackathon lab-protocol knowledge base and chatbot), scope, intended users, and known limitations.

2. Add `CONTRIBUTING.md` and `CODE_OF_CONDUCT.md` at the repository root to support open collaboration and set contributor expectations.
3. Pin dependency versions in `CODE/requirements.txt` (or add a lockfile / `environment.yml`) and document the supported Python version.
4. Document the provenance, source, and license of each stored protocol PDF/DOCX, and add a lightweight index describing how they feed the chatbot's document pipeline.
5. Add automated tests (e.g., `pytest`) for the `CODE/` modules and a CI workflow (e.g., GitHub Actions) that runs them on every push/PR.
6. Move (or add) a `LICENSE` file at the repository root so the whole repository's reuse terms are unambiguous, and add a `CITATION.cff`.
7. Add a brief ethics/accessibility/sustainability note (e.g., data handling expectations for lab protocols, plans for long-term maintenance).

Limitations

This snapshot only reflects file-presence and content inspection of the repository at the reviewed commit; it does not execute code, run tests, or verify the scientific accuracy of the stored lab protocols. It is not an audit of security, regulatory, or institutional compliance, and it is not an official Turing Way certification.